

Advanced Low-Power Systolic Array Architectures for Convolutional Neural Network Acceleration on Field-Programmable Gate Arrays

The rapid evolution of deep learning, particularly the emergence of complex Convolutional Neural Networks (CNNs) and Large Language Models (LLMs), has fundamentally transformed the requirements for high-performance computing. While the initial growth of these models was fueled by the massive parallelization capabilities of Graphics Processing Units (GPUs), the subsequent shift toward edge computing and embedded systems has exposed critical limitations in traditional hardware paradigms.¹ Modern applications in autonomous driving, real-time maritime perception, and near-sensor computing demand a sophisticated balance of high throughput and ultra-low power consumption that general-purpose processors struggle to provide.³ Within this context, Field-Programmable Gate Arrays (FPGAs) have emerged as a premier hardware platform, offering a unique blend of hardware-level reconfigurability and direct parallel execution.⁵ The most successful architectural strategy for deploying CNNs on FPGAs has been the adoption of systolic arrays. These grid-based arrangements of processing elements provide rhythmic data movement that minimizes global memory access, effectively mitigating the "memory wall" that plagues Von Neumann architectures.⁶ This report provides an exhaustive investigation into the design and optimization of low-power systolic arrays for CNN acceleration, encompassing architectural taxonomies, circuit-level power gating, algorithmic compression, and hardware-software co-design frameworks.

The Architectural Imperative for Reconfigurable AI Acceleration

The proliferation of artificial intelligence in embedded or mobile platforms has highlighted the critical role of specialized hardware. Researchers increasingly recognize that the efficiency of deep learning inference is not merely a software optimization problem but a fundamental hardware challenge.⁸ The computational overhead and high resource consumption of modern CNNs severely restrict their deployment on standard microprocessors.³ FPGAs address these issues by providing a flexible and reconfigurable fabric, allowing for model-specific customization that maintains high energy efficiency.¹ Unlike Application-Specific Integrated Circuits (ASICs), which offer high performance but remain inflexible after fabrication, FPGAs can be reprogrammed to adapt to evolving AI models, which is essential given the rapid pace of innovation in the field.¹

Traditional central processing units (CPUs) and GPUs rely on the Von Neumann architecture,

where instructions and data must be fetched from memory for every operation. This creates a bottleneck that limits scalability and increases power consumption due to constant data movement.⁷ In contrast, systolic arrays integrated into FPGA fabrics allow data to be fed into an array of processing elements (PEs) and propagated locally, reducing the frequency of power-intensive off-chip memory accesses.⁷ This spatial computing paradigm is particularly valuable for convolutional layers, which involve massive amounts of data reuse. By optimizing the dataflow and memory hierarchy, FPGAs can achieve a performance-per-watt that significantly outperforms general-purpose hardware.¹⁰

Hardware Platform	Flexibility	Latency	Energy Efficiency	Design Cycle
CPU	Very High	High	Low	N/A
GPU	High	Medium/Variable	Medium	Very Short
FPGA	High (Reconfigurable)	Low/Deterministic	High	Medium
ASIC	Low	Lowest	Very High	Long

Source: ¹

The shift toward FPGAs is further driven by the availability of sophisticated deep-learning-specific enhancements to traditional FPGA building blocks. Modern architectures are evolving to include specialized arithmetic circuitry, optimized logic blocks (LBs), and advanced on-chip memories that integrate directly into the fabric to accelerate tensor computations.⁵ These enhancements enable FPGAs to maintain a fraction of the development cost and time of custom silicon while delivering near-ASIC performance for specific edge workloads.⁵

Foundations of Systolic Array Design and Dataflow Optimization

A systolic array is defined as a spatial architecture composed of a grid of tightly coupled

processing elements that compute and exchange data with minimal global control.⁶ This "rhythmic" operation—analogue to the human circulatory system pumping blood—is what gives the architecture its name. In the context of CNNs, the systolic array is primarily used to accelerate matrix-matrix or matrix-vector multiplications, which form the core of convolutional and fully connected layers.⁷

Processing Element (PE) Microarchitecture

The processing element is the fundamental unit of the systolic array. Each PE typically consists of a multiplier, an accumulator, local registers for operand buffering, and routing logic for the movement of partial sums.⁶ In an FPGA implementation, these PEs can be synthesized using either Look-Up Tables (LUTs) or Digital Signal Processors (DSPs). Research indicates that while a single PE might require approximately 138 LUTs, it can be efficiently implemented using only two DSPs, making DSP-based synthesis a preferred method for maximizing hardware utilization.¹⁵

The internal logic of a PE is designed to support different arithmetic precisions, ranging from high-precision floating-point (FP32) to low-bit integers (INT8, INT4, or even binary).⁶ Modern variants of these PEs also include support for energy-efficient "approximate modes" and the ability to compute nonlinear activation functions directly within the array, further reducing the need to move data to external modules.⁶

Dataflow Taxonomies and Energy Efficiency

The "dataflow" of a systolic array refers to the specific pattern in which activations, weights, and partial sums move through the grid. Optimizing this dataflow is the primary mechanism for reducing energy consumption, as it determines the frequency of local data reuse versus global memory access.⁶

Weight-Stationary (WS) Dataflow In the Weight-Stationary paradigm, filter weights are pre-loaded into the registers of each PE and remain fixed for the duration of the computation. Input activations flow horizontally across the array, while partial sums are accumulated vertically.⁶ This dataflow is highly effective for CNNs because weights are reused repeatedly across multiple inputs. By keeping weights stationary, the design minimizes the power-intensive process of re-fetching filter parameters from memory.¹⁶ However, WS dataflow can suffer from high memory traffic if the convolution operation is not properly tiled or if the array must wait for a full weight-reload before starting the next operation.⁶ To mitigate this, advanced designs utilize double-buffering in each PE to store both current and next-layer weights, allowing for continuous computation.¹⁵

Output-Stationary (OS) Dataflow The Output-Stationary dataflow aims to minimize the movement of partial sums, which often have higher bit-widths than either weights or activations. In this configuration, each PE accumulates a complete output value locally before passing it out of the array.⁶ This approach is particularly beneficial for reducing the energy consumed by the

high-activity registers associated with accumulation.¹⁷

Row-Stationary (RS) Dataflow Row-Stationary dataflow is designed to maximize 2D reuse by streaming and partially caching both inputs and weights while accumulating partial sums locally.⁶ While RS can achieve the lowest total data movement, it often requires larger on-chip scratchpads (SRAMs), leading to higher area and energy costs per PE.⁶

Input-Stationary (IS) Dataflow In Input-Stationary designs, the input activations are fixed in the PEs while weights and partial sums move. This is less common for standard CNNs but is gaining popularity for Transformer models and Large Language Models (LLMs) where the input tokens (activations) are reused across massive weight matrices.⁷

Dataflow Paradigm	Stationary Operand	Primary Advantage	Typical Use Case
Weight-Stationary	Weights/Filters	Minimal weight fetching	CNNs with high filter reuse
Output-Stationary	Partial Sums	Minimal psum movement	High-precision accumulation
Row-Stationary	Local Rows	Maximum 2D reuse	Optimized local caching
Input-Stationary	Activations	Minimal activation fetching	Transformers/LLMs

Source: ⁶

Advanced Low-Power Circuit Design and Gating Techniques

While architectural choices define the broad energy profile of a systolic array, fine-grained circuit-level optimizations are required to meet the strict power constraints of edge devices. Research has focused on three primary areas: clock gating, power gating, and the use of approximate arithmetic.

Clock Gating (ECG, LECG, and LECE)

A significant portion of an FPGA's power consumption is dynamic, caused by the constant toggling of clock signals in logic blocks that may not be performing useful work.¹⁹ Clock gating (CG) is a technique that disables the clock signal to inactive modules, effectively "freezing" their state and eliminating switching power.¹⁹ This is particularly relevant for CNNs, where many PEs may remain in standby states depending on the sparsity of the input data or the specific tiling strategy.¹⁹

Three specific clock gating techniques have shown significant promise for FPGA-based CNN accelerators:

1. **Local Explicit Clock Enable (LECE):** This method updates the output on the clock's rising edge only when a high-enable signal is present. It provides precise control over individual registers, which is critical for memory modules like Dual-Port RAM.¹⁹
2. **Local Explicit Clock Gating (LECG):** LECG optimizes power by updating all outputs simultaneously when triggered by an enable signal. It utilizes XOR gates and latches to prevent toggling glitches, ensuring that the clock activates only when necessary.¹⁹
3. **Enhanced Clock Gating (ECG):** This technique uses XOR gates to control both input clock and enable signals for multibit I/O data. Implementation of ECG in the Multiply-Accumulate (MAC) modules of a systolic array on a PYNQ-Z2 board resulted in a total dynamic power consumption of 1.440 W while performing 6.97 billion operations.¹⁹

The combined application of LECG and LECE can lead to a 22% decrease in dynamic power and a 25% reduction in total on-chip power compared to non-gated designs.¹⁹

Fine-Grained Power Gating (PI-SA)

Unlike clock gating, which reduces dynamic power, power gating aims to reduce static (leakage) power by completely shutting off the power supply to idle components. The Power-Intent Systolic Array (PI-SA) is a novel micro-architecture that uses the industry-standard Unified Power Format (UPF) to define switchable power domains.¹⁴

A key innovation in PI-SA is the bit-level decomposition of the multiplier. Instead of a single 8-bit multiplier, the PE uses four 4-bit sub-multipliers. This allows the system to shut off specific sub-multipliers when processing operands with leading zeros—a common occurrence in quantized networks.¹⁴ By replacing traditional Carry Propagation Adders (CPAs) with Carry Save Adders (CSAs) and optimizing the reduction tree (reducing XOR gates from six to two), PI-SA architectures have demonstrated power reductions of up to 57% and area improvements of 32%.¹⁴

Approximate Computing and Arithmetic Efficiency

In many neural network applications, absolute arithmetic precision is not required to maintain high inference accuracy. Approximate computing leverages this error resilience to simplify

hardware logic, thereby reducing power and area.²⁰

Recursive and Truncated Multipliers Approximate multipliers can be constructed by pruning the hardware resources that form the least significant columns of the partial product tree.²¹ For an 8x8 multiplier, removing the AND gates and compressors for the seven least-significant bits transforms the exact accumulation tree into an approximate one, drastically reducing logic density.²¹ Recursive multipliers take this further by composing large multipliers from smaller, inaccurate building blocks. The error induced by these approximations, denoted as

$\epsilon = W \cdot A - AMR(W, A)$, can often be mitigated at runtime using "control variate" techniques that estimate and subtract the error without requiring model retraining.²¹

Hardware-Efficient Cells (PPC and NPPC) Researchers have proposed novel PE designs using Positive Partial Product (PPC) and Negative Partial Product (NPPC) cells. These 8-bit exact and approximate PE designs have achieved energy savings of 22% and 32%, respectively, when integrated into 8x8 systolic arrays for applications like edge detection and CNN inference.²² In some instances, the proposed approximate multipliers have demonstrated up to 38.75% power improvement over standard Xilinx exact multipliers.²⁰

Spatiotemporal Serial Processing and Bit-Serial Architectures

Most conventional systolic arrays, such as the Google TPU, utilize bit-parallel processing, where all bits of an operand are processed simultaneously. While high in throughput, this leads to significant power density and thermal challenges.¹⁷ An alternative approach is spatiotemporal serial processing, as seen in the Serial/Parallel Systolic Array (SPSA) and Octet Serial/Parallel Systolic Array (OSPSA).¹⁷

SPSA and OSPSA Mechanisms

In an SPSA PE, weights are maintained in bit-parallel format, but activations arrive bit-serially over an 8-cycle loop, starting from the Most Significant Bit (MSB). This allows for dynamic precision adjustment—if a model only needs 4 bits of precision for a specific layer, the hardware can stop after 4 cycles, effectively doubling the energy efficiency.¹⁷

OSPSA improves upon SPSA by processing a column of eight concurrent bits from separate activations simultaneously. This maintains the throughput of a bit-parallel design while retaining the energy-saving benefits of serial processing, such as "bit-column-wise zero skipping".¹⁷ Because serial PEs have a simpler physical structure, they can often operate at higher clock frequencies than their parallel counterparts while consuming fewer FPGA resources.¹⁷

Feature	Bit-Parallel SA (PSA)	Bit-Serial SA (SPSA/OSPSA)

Data Arrival	Simultaneous (All bits)	Sequential (Bit-by-bit)
Precision	Fixed (e.g., INT8)	Dynamic/Layer-wise
Logic Complexity	High (Complex Multipliers)	Low (AND-gate based)
Energy Saving	Limited	High (via Zero skipping)
Throughput	High	Variable (scales with precision)

Source: ¹⁷

Hardware-Aware Model Compression: Quantization and Pruning

The most effective way to design a low-power systolic array is to reduce the workload before it ever reaches the hardware. Model compression—specifically quantization and pruning—allows for a radical reduction in the number of required operations and memory accesses.²³

Quantization Strategies

Quantization involves reducing the bit-width of weights and activations from FP32 to low-bit integers like INT8, INT4, or even 1-bit binary values.²⁵ INT8 quantization is widely adopted as it offers a 4x reduction in memory and 16x improvement in energy efficiency with negligible accuracy loss.²⁵ For extreme constraints, binary and ternary networks encode values as two or three states, replacing multiplications with simple logic gates or "XNOR-popcount" operations.²⁵

Intra-Layer Mixed Precision (FILM-QNN) A common challenge in mixed-precision hardware is the idling of resources. In an inter-layer mixed-precision design, a high-precision layer might saturate the DSPs while the LUTs remain idle, and a low-precision layer might do the opposite.²⁶ The FILM-QNN framework solves this by mixing precision *within* each layer. For example, 95% of weights are quantized to 4-bit (processed in LUTs) and 5% to 8-bit (processed in DSPs).²⁶ This allows the hardware to utilize both DSPs and LUTs concurrently, increasing throughput by 1.71x to 2.95x while maintaining accuracy levels of pure 8-bit designs.²⁶

Hardware-Aware INT8 Quantization Research on MobileNetV2 deployment on the AMD/Xilinx Kria KV260 FPGA has shown that Quantization-Aware Training (QAT) is superior to Post-Training Quantization (PTQ) for maintaining accuracy (91.74% vs 90.94% relative to a 92.42% FP32 baseline).²⁷ The resulting hardware implementation achieved 30 FPS at only 3 W of power.²⁷

Pruning and Sparsity Exploitation

Pruning removes redundant weights from a network, creating a "sparse" matrix where many entries are zero.²⁴ While unstructured pruning achieves the highest compression rates, it creates irregular data patterns that are difficult to accelerate.²⁸

Systolic Arrays and Structured Pruning (SASP) The SASP framework proposes matching the size of pruned weight blocks with the physical dimensions of the systolic array mesh.²⁸ If a weight tile matches the array size and is found to be zero, the entire computation for that tile can be skipped. This "tile-skipping" avoids the control logic overhead of fine-grained zero-skipping and has resulted in system-wide speedups of up to 44% and energy savings of 42%.²⁸

Group Sparse Regularization (GSR) To address internal buffer misalignments and load imbalances caused by random pruning, researchers use group pruning with GSR. This ensures that the pruned CNN remains structurally consistent with the systolic dataflow. Implementations of VGG16 on a Xilinx VC707 FPGA using this method achieved 188.41 GOPS at 100 MHz, consuming 8.15 W.³⁰

Memory Hierarchy and Efficient Data Orchestration

The high computational density of a systolic array is useless if the processing elements cannot be supplied with data fast enough. The "memory wall" is a primary bottleneck in CNN accelerators, particularly for edge devices with limited bandwidth.²⁴

On-Chip Buffer Management

Energy-efficient systolic arrays utilize a multi-level storage structure combining Block RAM (BRAM), registers, and SRAM.⁹ On-chip BRAMs are used to provide predictable, multi-ported buffering for streaming dataflows. For instance, a maritime perception CNN on a Zynq-7020 utilized 58 BRAM36 blocks (2.04 Mbit) to implement line-buffered convolutions, enabling low-latency processing without frequent DDR accesses.⁴

Efficient data reuse is achieved by buffering image values and filter weights locally. The unique dataflow of the systolic array reduces the data movement from off-chip DRAM to on-chip buffers, which not only saves energy but also increases the overall throughput.¹⁵

AXI-Stream and DMA-Based Movement

Data movement between the external DDR memory and the programmable logic is typically managed through an AXI DMA engine connected to a high-performance (HP) AXI port. The

compute blocks themselves are interconnected using AXI-Stream interfaces, which support deterministic dataflows and reduce the overhead of complex memory management.⁴ This hardware-software partitioning—where the FPGA hosts compute-intensive paths and an ARM processing system (PS) handles lightweight control—is standard for Modern System-on-Chip (SoC) FPGAs.⁴

Toolchains, Synthesis, and Power Profiling Methodologies

The design of a low-power systolic array requires a sophisticated toolchain that spans from high-level algorithms to low-level hardware verification.

High-Level Synthesis (HLS) vs. RTL

High-Level Synthesis (HLS) tools, such as Xilinx Vitis HLS or Intel Quartus HLS, allow designers to create FPGA circuits using C/C++ or SystemC.³² While HLS accelerates the design cycle, it often generates "closed" RTL that is difficult to modify for manual power optimizations like fine-grained clock gating.¹⁹ Consequently, some researchers prefer using hardware description languages (Verilog/VHDL) or "Verilog generators" like Spinal HDL, which allow for more intuitive descriptions of digital circuits while maintaining high maintainability.³⁴

Power Analysis and Estimation

Accurate power profiling is essential to verify that the systolic array meets its low-power requirements.

AMD/Xilinx Vivado Flow

- **Xilinx Power Estimator (XPE) / Power Design Manager (PDM):** These spreadsheet-based tools estimate power at any stage of the design cycle using simple wizards to analyze resource counts and expected toggle rates.³⁵
- **Vivado Power Analyzer:** This tool performs post-routing analysis, reading exact logic and routing resources from the implemented design. For maximum accuracy, designers use a Switching Activity Interchange Format (SAIF) file generated from post-synthesis functional simulation to provide realistic switching activities.³⁶

Intel/Altera Quartus Flow

- **Power and Thermal Calculator (PTC):** PTC allows for quick estimation of power and thermal parameters before, during, and after the design phase for Agilex and Stratix 10 devices.³⁸
- **Quartus Prime Power Analyzer (QPA):** QPA produces the most accurate results for post-fit designs by using detailed routing and configuration information.⁴⁰

Comparative Analysis of Performance and Energy Efficiency

To evaluate the success of these low-power techniques, we must examine the performance metrics of real-world systolic array implementations across various FPGA platforms.

Metrics of Efficiency (GOPS/W)

The standard metric for evaluating AI accelerators is the energy efficiency ratio, measured in Giga Operations Per Second per Watt (GOPS/W) or Tera Operations Per Second per Watt (TOPS/W). While traditional CPU and GPU deployments struggle to exceed a few GOPS/W, optimized FPGA systolic arrays can reach significantly higher values.³

Accelerator Architecture	Platform	Throughput (GOPS)	Power (W)	Efficiency (GOPS/W)
Fused Operator SA	Nexys Video	20.6	1.96	10.46
LECG + LECE Optimized	PYNQ-Z2	N/A	1.33	43.9
FILM-QNN (Mixed-Bit)	ZCU102	N/A	N/A	89.4 (est.)
INT8 SIMD Engine	ZCU102	1,708	19.1	89.4
INT8 SIMD SA	Alveo U250	17,300	118	146.6
Low-Bit LLM SA (3-bit)	Alveo U250	13,568	61.8	219.4

Group Pruned SA	VC707	188.4	8.15	23.1
------------------------	-------	-------	------	------

Source: ³

Research on a 3-bit integerized model validated on a 16nm Alveo U250 FPGA delivered 13,568 GOPs/s at an efficiency of 219.4 GOPs/W. Compared to a GPU of the same technology (GTX 1080), this design offered 4.47x better power efficiency.¹⁸ Even compared to cutting-edge GPUs like the RTX 5090, the FPGA-based systolic array achieved 20% better power efficiency despite having lower absolute throughput.¹⁸

Second-Order Insights: Accuracy vs. Power

A deeper analysis of the data suggests that the "sweet spot" for edge AI deployment is shifting toward 3-bit and 4-bit quantization combined with structured N:M pruning. While binarization offers the highest power savings, the resulting accuracy loss often makes it unsuitable for complex vision tasks. However, 3-bit networks have been shown to maintain over 96% accuracy on CIFAR-10 and nearly 78% on ImageNet while providing superior hardware performance compared to 8-bit implementations.¹⁸

Furthermore, the integration of operator fusion (merging Conv, BN, and Activation layers) not only speeds up computation but also reduces the number of intermediate feature map writes to BRAM, leading to an energy efficiency ratio that is 58–350% higher than existing mainstream accelerators.³

Advanced and Emerging Systolic Array Paradigms

As CNN models evolve toward Transformers and beyond, systolic array architectures are also advancing to handle more irregular workloads.

3D Systolic Arrays

Traditional 2D systolic arrays often face scalability issues and limitations in processing speed for high-resolution images. Emerging 3D systolic array architectures process data in three dimensions, improving both area utilization and operational speed.⁴² By feeding input data in the X-Y plane and filter coefficients through the Z-X plane, 3D arrays can more efficiently handle combined convolution and max-pooling layers in a single framework, significantly reducing overall processing time.⁴²

Sparsity-Aware and Near-Cache Architectures

Next-generation accelerators like SPARK are moving toward sparsity-aware, near-cache architectures. These systems reconfigure existing CPU L1 caches to perform acceleration,

minimizing data movement overheads. SPARK has shown 152x to 740x energy improvements over traditional CPUs for sparse real-life workloads.⁴³ Similarly, SuperNoVA enables high-accuracy SLAM (Simultaneous Localization and Mapping) in resource-constrained settings by virtualizing hardware resources for efficient graph processing within the systolic array backend.⁴⁴

Conclusions and Strategic Recommendations

The research presented in this report underscores the critical importance of a multi-layered approach to low-power systolic array design. Achieving the high energy efficiency required for edge AI deployment is not the result of a single optimization but rather the synergy of architectural, circuit-level, and algorithmic strategies.

The transition from traditional bit-parallel architectures to bit-serial and approximate computing modes offers a viable pathway to reducing power consumption by over 50%.¹⁴ Furthermore, the adoption of hardware-aware model compression—specifically intra-layer mixed-precision quantization and systolic-friendly structured pruning—ensures that the underlying FPGA resources are utilized at their theoretical maximum.²⁶

Key recommendations for the development of final-semester thesis projects in this domain include:

1. **Prioritize Software-Hardware Co-Design:** Focus on matching pruning block sizes to systolic mesh dimensions (SASP) to enable efficient tile-skipping without complex control logic.²⁸
2. **Implement Fine-Grained Gating:** Utilize LECG and LECE at the RTL level, particularly within the MAC and Dual-Port RAM modules, to achieve immediate power reductions of up to 25%.¹⁹
3. **Explore Low-Bit Precision:** Investigate the trade-offs of 3-bit and 4-bit integerization, which provide significantly better GOPS/W than INT8 while maintaining competitive accuracy for vision and language tasks.¹⁸
4. **Leverage Advanced Toolchains:** Use power profiling tools like Vivado Power Analyzer with SAIF vectors early in the design cycle to identify and mitigate high-power-consuming hierarchies.³⁶

By embracing these sophisticated design principles, researchers and engineers can continue to push the boundaries of what is possible for artificial intelligence in embedded, mobile, and autonomous systems, ensuring that high-performance AI remains sustainable and energy-efficient in the years to come.¹

Works cited

1. Architectural Design and Performance Analysis of FPGA based AI Accelerators: A Comprehensive Review - arXiv, accessed April 13, 2026,

- <https://arxiv.org/html/2603.08740v1>
2. FPGA-Based Deep Learning Acceleration: Analysis of Dataflow and Tiling Strategies, accessed April 13, 2026, <https://drpress.org/ojs/index.php/ajst/article/view/34459>
 3. Architecture Design of a Convolutional Neural Network Accelerator for Heterogeneous Computing Based on a Fused Systolic Array - PMC, accessed April 13, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12845848/>
 4. FPGA-Based CNN Acceleration on Zynq-7020 for Embedded Ship ..., accessed April 13, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12986546/>
 5. Field-Programmable Gate Array Architecture for Deep Learning: Survey and Future Directions - IEEE Xplore, accessed April 13, 2026, <https://ieeexplore.ieee.org/iel8/5/11313344/11222684.pdf>
 6. Systolic Array Architectures - Emergent Mind, accessed April 13, 2026, <https://www.emergentmind.com/topics/systolic-array-sa-architectures>
 7. Systolic Array Data Flows for Efficient Matrix Multiplication in Deep Neural Networks - arXiv, accessed April 13, 2026, <https://arxiv.org/html/2410.22595v1>
 8. A Systolic Array-based General Convolutional Neural Network Accelerator - Diva-portal.org, accessed April 13, 2026, <https://www.diva-portal.org/smash/get/diva2:1942273/FULLTEXT02.pdf>
 9. An Energy-Efficient Convolutional Neural Network Processor Architecture Based on a Systolic Array - MDPI, accessed April 13, 2026, <https://www.mdpi.com/2076-3417/12/24/12633>
 10. Low-Power FPGA Design for AI Acceleration in Edge Devices - ResearchGate, accessed April 13, 2026, https://www.researchgate.net/publication/388864243_Low-Power_FPGA_Design_for_AI_Acceleration_in_Edge_Devices
 11. Efficient SIMD computations on FPGA: Architectures, design techniques, and applications - World Journal of Advanced Research and Reviews, accessed April 13, 2026, https://wjarr.com/sites/default/files/fulltext_pdf/WJARR-2025-1056.pdf
 12. A Component-Centric Perspective on Hardware Accelerators for LLMs - IEEE Xplore, accessed April 13, 2026, <https://ieeexplore.ieee.org/iel8/6287639/6514899/11164709.pdf>
 13. Embedded Intelligence on FPGA: Survey, Applications and Challenges - MDPI, accessed April 13, 2026, <https://www.mdpi.com/2079-9292/10/8/895>
 14. Power-Intent Systolic Array Using Modified Parallel Multiplier for ..., accessed April 13, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10181616/>
 15. FPGA Group6 Final Project - Systolic Array as an CNN Accelerator - HackMD, accessed April 13, 2026, <https://hackmd.io/@fpga/SyV8U2GD3>
 16. AI Architecture 14. Dataflow Taxonomy: TPU vs Output Stationary vs ..., accessed April 13, 2026, <https://rtlearner.com/en/ai-architecture-14-dataflow-taxonomy-ws-os-rs/>
 17. Enhancing Computation-Efficiency of Deep Neural Network ... - MDPI, accessed April 13, 2026, <https://www.mdpi.com/2504-4990/6/3/70>
 18. Systolic Array-based Architecture for Low-Bit Integerized Vision Transformers - arXiv, accessed April 13, 2026, <https://arxiv.org/html/2508.20334v1>

19. A High-Performance and Ultra-Low-Power Accelerator Design for ..., accessed April 13, 2026, <https://www.mdpi.com/2079-9292/13/13/2676>
20. Design of a Configurable Multifunctional Systolic Array Based on FPGA - ResearchGate, accessed April 13, 2026, https://www.researchgate.net/publication/401289144_Design_of_a_Configurable_Multifunctional_Systolic_Array_Based_on_FPGA
21. Leveraging Highly Approximated Multipliers in DNN Inference - IEEE Xplore, accessed April 13, 2026, <https://ieeexplore.ieee.org/iel8/6287639/10820123/10924159.pdf>
22. Energy Efficient Exact and Approximate Systolic Array Architecture for Matrix Multiplication, accessed April 13, 2026, https://www.researchgate.net/publication/395212398_Energy_Efficient_Exact_and_Approximate_Systolic_Array_Architecture_for_Matrix_Multiplication
23. (PDF) FPGA Co-Design for Efficient N:M Sparse and Quantized Model Inference, accessed April 13, 2026, https://www.researchgate.net/publication/399275989_FPGA_Co-Design_for_Efficient_NM_Sparse_and_Quantized_Model_Inference
24. A Hardware-Friendly High-Precision CNN Pruning Method and Its FPGA Implementation, accessed April 13, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9862432/>
25. A Survey on Binary and Ternary Neural Networks and Their Realization in Compute-in-Memory for Edge Intelligence - IEEE Xplore, accessed April 13, 2026, <https://ieeexplore.ieee.org/iel8/6488907/6702522/11250544.pdf>
26. FILM-QNN: Efficient FPGA Acceleration of Deep Neural Networks ..., accessed April 13, 2026, <https://www.sfu.ca/~zhenman/files/C23-FPGA2022-FILM-QNN.pdf>
27. (PDF) Hardware-Aware INT8 Quantization and FPGA Deployment of MobileNetV2 for Real-Time Facial Landmark Detection - ResearchGate, accessed April 13, 2026, https://www.researchgate.net/publication/403136468_Hardware-Aware_INT8_Quantization_and_FPGA_Deployment_of_MobileNetV2_for_Real-Time_Facial_Landmark_Detection
28. Systolic Arrays and Structured Pruning Co-design for Efficient Transformers in Edge Systems - arXiv, accessed April 13, 2026, <https://arxiv.org/html/2411.10285v1>
29. [2411.10285] Systolic Arrays and Structured Pruning Co-design for Efficient Transformers in Edge Systems - arXiv, accessed April 13, 2026, <https://arxiv.org/abs/2411.10285>
30. An Energy-Efficient Implementation of Group Pruned CNNs on FPGA - SciSpace, accessed April 13, 2026, <https://scispace.com/pdf/an-energy-efficient-implementation-of-group-pruned-cnns-on-28wbodk5m8.pdf>
31. Adaptive Global Power-of-Two Ternary Quantization Algorithm Based on Unfixed Boundary Thresholds - PMC, accessed April 13, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10781396/>
32. (PDF) A Survey and Evaluation of FPGA High-Level Synthesis Tools -

- ResearchGate, accessed April 13, 2026,
https://www.researchgate.net/publication/288872739_A_Survey_and_Evaluation_of_FPGA_High-Level_Synthesis_Tools
33. A Survey and Evaluation of FPGA High-Level Synthesis Tools - Semantic Scholar, accessed April 13, 2026,
<https://www.semanticscholar.org/paper/A-Survey-and-Evaluation-of-FPGA-High-Level-Tools-Nane-Sima/dcf07215bd3775967d11b07a925f29ab0ba25d4f>
 34. Agile Design and Implementation of a Systolic Array-Based CNN Accelerator - IJERM, accessed April 13, 2026,
https://www.ijerm.com/download_data/IJERM1103012.pdf
 35. Xilinx Power Estimator (XPE) - AMD, accessed April 13, 2026,
<https://www.amd.com/en/products/adaptive-socs-and-fpgas/technologies/power-efficiency/power-estimator.html>
 36. Vivado Power Analysis - 2025.2 English - UG907, accessed April 13, 2026,
<https://docs.amd.com/r/en-US/ug907-vivado-power-analysis-optimization/Vivado-Power-Analysis>
 37. 32dynamic Power Estimation Using Vivado Power Report | PDF | Computers - Scribd, accessed April 13, 2026,
<https://www.scribd.com/document/621569300/32Dynamic-Power-Estimation-Using-Vivado-Power-Report>
 38. Let's Try Agilex™ 3 - Estimation of power consumption – Macnica Altera FPGA Insights, accessed April 13, 2026,
<https://malt.zendesk.com/hc/en-us/articles/50434254299929-Let-s-Try-Agilex-3-Estimation-of-power-consumption>
 39. How to Estimate Altera® FPGA Power Consumption - Semiconductor Business -Macnica, accessed April 13, 2026,
<https://www.macnica.co.jp/en/business/semiconductor/articles/altera/121893/>
 40. Early Power Estimators (EPE) and Power Analyzer - Altera, accessed April 13, 2026,
<https://www.altera.com/design/resource/power/epe-and-pa>
 41. 4.1. Power Estimation and Analysis Tools - Intel, accessed April 13, 2026,
<https://www.intel.com/content/www/us/en/docs/programmable/814028/current/power-estimation-and-analysis-tools.html>
 42. FPGA Implementation of High-Speed Accelerators for CNN Architecture Using 3D Systolic Array Architecture - IEEE Xplore, accessed April 13, 2026,
<https://ieeexplore.ieee.org/iel8/10939028/10939055/10939284.pdf>
 43. SPARK: Sparsity Aware, Low Area, Energy-Efficient, Near-memory Architecture for Accelerating Linear Programming Problems, accessed April 13, 2026,
https://lca.ece.utexas.edu/pubs/SPARK_HPCA_2025.pdf
 44. SuperNoVA: Algorithm-Hardware Co-Design for Resource-Aware SLAM - People @EECS, accessed April 13, 2026,
<https://people.eecs.berkeley.edu/~ysshao/assets/papers/supernova-asplos2025.pdf>
 45. hardware-software-co-design-for-power-efficient-edge-ai-systems - URF Publishers, accessed April 13, 2026,
<https://urfpublishers.com/journal/artificial-intelligence/article/view/hardware-software-co-design-for-power-efficient-edge-ai-systems>

[e-co-design-for-power-efficient-edge-ai-systems](#)